AFSAL A AZEEZ

Full-Stack & AI Engineer • GenAI • IIT Bombay M.Tech

afsalaazeez@gmail.com • +971 55 388 5320
Abu Dhabi, UAE • **Available Immediately**
linkedin.com/in/afsalaazeez • github.com/zencodelab • afsal-a-azeez.in

Professional Summary

Full-Stack & AI Engineer with 5+ years building production software at a Mayfield-funded DevOps & AI startup — led a frontend team of 4 engineers, owned RoostGPT's Java unit test generation pipeline end-to-end (93% compile success rate) and built its test scenario analysis feature (4× test coverage), multi-LLM provider UI, Git platform integrations and Microsoft Teams bot. Since July 2026, built Markaba AI with Claude Code — an AI automotive diagnostics and marketplace platform taken from first commit to live production in two weeks, then extended with a natural-language booking assistant, a golden-set LLM eval harness and two live MCP connectors. IIT Bombay M.Tech (GATE AIR-241); fully attested academic credentials for immediate UAE residency compliance.

Technical Skills

- **Architecture & Design:** Microservices, Distributed Systems, RESTful/GraphQL APIs, Serverless, Event-Driven Architecture, System Design.
- **Frontend Engineering:** React.js, Next.js, Electron.js, Redux Toolkit, Zustand, TailwindCSS, SCSS, HTML5/CSS3.
- **Backend Engineering:** Node.js (TypeScript), Python (FastAPI), Express, Fastify, OAuth, JWT.
- **AI & LLM:** GPT-4, Gemini, Claude, Llama 3, LangChain, LlamaIndex, OpenAI API, RAG Pipelines, Vector Databases, Tool-Calling Agents, Model Context Protocol (MCP), LLM Evaluation, Prompt Caching, Multi-LLM Orchestration, AI Agent Orchestration, AI-Assisted Development (Claude Code).
- **DevOps & Cloud:** AWS (EC2, S3, Lambda, EKS), Docker, Docker Compose, Kubernetes, OpenShift, Helm, Caddy, GitHub Actions, Jenkins.
- **Databases & Caching:** PostgreSQL, MySQL, MongoDB, Redis, SQLite, Elasticsearch.
- **Testing & Security:** Playwright, OWASP ZAP, Postman, JUnit, pytest, Jest.
- **Version Control & Platforms:** GitHub, GitLab, Bitbucket, Azure Repos, Microsoft Teams (Bot Framework).

Professional Experience

Founder & Lead Engineer **Jul 2026 – Present**

Full-Stack & AI Engineer *Markaba AI (markabaai.com) — AI Automotive Diagnostics Platform*

Built with Claude Code as the primary coding tool; owned the requirements, product and design decisions.

- **AI Diagnosis Engine:** Architected a multi-layer automotive intelligence engine (Diagnose → Match → Rank → Quote) converting natural-language vehicle symptoms into matched parts, ranked workshops and bookable quotes; runs on Claude in production (eval-validated Gemini adapter, switchable at runtime) behind a circuit breaker, Claude prompt caching (42% lower cost on a live eval run), and a daily AI-spend cap.
- **AI Booking Assistant:** Customers who name a garage can book it in plain language: the LLM extracts a structured request, and the backend resolves provider and slot against live data, prices it server-side, and asks rather than guesses when anything is ambiguous; the customer confirms via the existing booking endpoint, and it offers to book only when asked (eval-checked).
- **Tool-Calling Agent & Evals:** Built a 33-case golden-set eval harness (diagnosis, photo ID, voice; incl. Arabic), used on diagnosis to compare Claude vs Gemini, and A/B-tested a bounded tool-calling agent (5 tools; turn, tool-call, and timeout caps) against the production pipeline — no measured gain on 4 ambiguous cases at 2× latency and 3.6× cost, so it stays admin-only.
- **MCP Connectors (live in production):** Two Model Context Protocol servers letting Claude read Markaba AI — car-owner (7 read-only tools) and business (5) — OAuth resource servers with Clerk as the authorization server, prompt-injection mitigations for third-party text, per-tool output allowlists, and rate limits that fail closed on data calls; on for every account since Sep 2026.
- **Semantic Parts Matching (built; live site runs keyword search):** Hybrid parts search combining LlamaIndex/Ollama embeddings in pgvector with Postgres full-text search; production serves the full-text path.
- **Full-Stack Marketplace:** Shipped a bilingual (English/Arabic, RTL) React + Vite + TypeScript + Tailwind frontend and a FastAPI + SQLAlchemy + Alembic backend covering a parts marketplace, "Mod Discovery" upgrades, service booking, a "My Garage" vehicle-profile system, and provider and supplier portals with wholesale procurement; wrapped with Capacitor for Android (in Google Play closed testing) and iOS (TestFlight).
- **Parts Network (built; pilot pending):** B2B layer where workshops trade parts and quote repairs — part requests ranked by location and price, repair plans quoted by up to three workshops, and accepted jobs that book a

slot and raise one purchase order per seller. Forward-only Postgres state machines with arq workers, not LangGraph (the edges are business rules), plus a bilingual part-identity resolver where the model only suggests (61-case eval).

- **Sign-in & Notifications:** Built Clerk/OIDC sign-in (live since launch) and a WhatsApp Accept/Decline booking flow for partner workshops (Twilio sandbox), with each notification event written to an outbox in the same transaction as the booking change and sent by a background worker.
- **Production Delivery:** Took the product from first commit to live production in two weeks, owning the architecture and DevOps decisions — Docker and Caddy on a VPS with GitHub Actions CI; since added a one-command, health-gated deploy script with rollback, local k6 load tests, two backend replicas with rolling deploys, Postgres advisory locks, and encrypted off-box backups.
- **Tech Stack:** Python, FastAPI, React, Vite, TypeScript, Tailwind CSS, Capacitor, PostgreSQL, pgvector, Redis, arq, LlamaIndex, Docker, GitHub Actions, k6, Anthropic Claude API, Google Gemini API.

Member of Technical Staff **Sep 2020 – May 2026**

Full-Stack & AI Engineer *Zettabytes Inc. (Roost.ai) — Mayfield-funded DevOps & AI Startup · Remote / US-Based Market*

- **Team Leadership:** Led a frontend team of 4 engineers across both product phases below.

Phase B — RoostGPT: AI Test Generation Platform ***2023 – 2026***

- **RoostGPT — AI Testing Platform:** Built and shipped **RoostGPT**, an AI testing copilot leveraging LLMs (GPT-4, Gemini, Claude, Llama 3) to auto-generate unit, API, UI, and integration test cases from source code, Swagger/OpenAPI specs, and Jira user stories.
- **Microsoft Teams Notification Bot:** Took **end-to-end ownership** of building a Microsoft Teams notification bot for RoostGPT, delivering real-time test generation status updates, results summaries, and CI/CD pipeline notifications directly within Teams channels.
- **Git Platform Integrations:** Built integrations within RoostGPT connecting to GitHub, GitLab, Bitbucket, and Azure Repos for automated repository access, test generation triggers, and direct test commit workflows. Implemented GitHub pull request webhook hooks to automatically retrigger test generation on new code changes.
- **Multi-LLM Provider UI:** Built the frontend dashboard for configuring and switching between LLM providers — supporting OpenAI, Google Gemini, Anthropic Claude, and Meta Llama 3 as interchangeable backends, enabling customers to select their preferred AI model for test generation.
- **Java Unit Test Generation — End-to-End Ownership:** Owned the complete Java unit test generation pipeline — from repository ingestion through to final generated test files. Engineered LLM context by working directly on the **Spring Boot Java AST parsing service** to extract and structure class graphs, bean dependency trees, and method signatures from multi-module Maven/Gradle repositories. Diagnosed and resolved prompt engineering issues causing incorrect imports and missing mocks, achieving a **93% compile success rate** across generated test suites.
- **Test Scenario Analysis & Report UI:** Built the end-to-end test scenario analysis feature (frontend, API, and backend) — listing all generated tests with covered and uncovered scenarios, allowing users to select specific scenarios and trigger targeted test generation. Displayed generation summaries, compile check results, and parsed coverage report output, giving developers a full view of test quality before review and commit. Achieved **4x improvement in test coverage** and **85% reduction in manual testing time**.
- **Prompt Management System:** Designed and shipped a **nested-map data structure** for managing LLM prompts across multiple dimensions — model (GPT-4, Gemini, Claude, Llama 3), test type (unit, API spec, integration), and client. Implemented a deterministic lookup algorithm enabling the system to fetch the exact prompt for any combination at runtime, and enforced structured folder conventions for prompt files with **AWS S3-backed storage** for versioning and distribution.
- **Java AST Schema Redesign:** Flattened nested class structures into first-class entities, cutting per-class file size from **316KB to 17KB** (95% reduction) and improving LLM context efficiency; validated on ask-java and adk-java.
- **FQN Mapping & Method Overload Resolution:** Implemented Fully Qualified Name mapping for internal method calls, enabling accurate method overload identification via argument lists.
- **File-to-Class Entity Migration:** Led migration from file-based to class-based entity handling across the test generation pipeline, enabling AI agents to correctly receive structured parameters for Java unit test generation.
- **AST Output Optimisation:** Removed unnecessary body fields from method call extraction, reducing codeparse output from **15.9MB to significantly smaller payloads** — cutting LLM token costs and improving throughput.
- **Test Generation UI:** Built the full frontend for test generation workflows — repository analysis views, trigger dashboards, and review/refinement interfaces for inspecting, editing, and approving AI-generated tests.
- **User Access Management UI:** Built team-level permissions for test generation across repositories and projects.
- **AI Response Caching:** Designed an SQLite-based caching layer for LLM responses to eliminate redundant API calls, reducing cost and latency for repeated prompts.
- **Playwright E2E Testing:** Wrote and reviewed Playwright end-to-end test suites and built an API test analysis

page for scenario-specific execution tracking.

- **Technical Mentorship:** Mentored junior engineers and interns; conducted code reviews and tracked delivery via JIRA.
- **Tech Stack:** React.js, Next.js, TypeScript, Node.js, Spring Boot (Java), SQLite, REST APIs, LLM APIs (OpenAI, Gemini, Claude, Llama 3), Docker, GitHub Actions.

Phase A — Roost Desktop, Serverless & EaaS Platform ***2020 – 2022***

- **Roost Desktop Application:** Built and maintained a **cross-platform Electron.js + React.js** desktop application for DevOps collaboration targeting macOS, Windows, and Linux.
- **Kubernetes Visualisation:** Developed frontend UI for visualising local Kubernetes deployment environments — pod network topology, sidecar/service-mesh connections, and real-time cluster state monitoring.
- **Serverless Migration:** Transitioned the Roost Desktop product to a **serverless cloud architecture**, building the Roost Web UI as a browser-based interface extending the desktop app's capabilities.
- **Collaboration Features:** Implemented a collaboration layer enabling developers to share environment snapshots and deployment states with QA, product, and external stakeholders for review.
- **EaaS Feature Development:** Implemented multi-repo commit (Save All), repo-level config saving, environment variable injection, Helm chart filtering, CloudFormation (S3/JSON) integration, and Terraform UI with full API integration.
- **GitHub API Integration:** Built integrations with GitHub API for repository file discovery, deployment config parsing, and Kubernetes/Helm YAML filtering for the EaaS config workflow.
- **Slack & JIRA Config UI:** Added Slack token/channel suggestion dropdowns and JIRA-referenced feature tracking fields across the EaaS configuration UI.
- **Enterprise Deployments:** Supported on-premise and OpenShift deployments for enterprise clients across the US market, collaborating directly with their engineering and DevOps teams on cluster integration and platform rollout.
- **Tech Stack:** React.js, Node.js, Python, Go, REST/GraphQL APIs, Kubernetes, Docker, Terraform, AWS (EKS, EC2, S3, Lambda), Electron.js.

Education

Master of Technology (M.Tech.) in Computer Science & Engineering **Graduated 2020**

Indian Institute of Technology (IIT), Bombay *Mumbai, India*

- GATE AIR-241 — Top 0.22% out of 107,893 candidates. Degree and academic credentials fully attested for UAE international visa and residency compliance.

Bachelor of Technology (B.Tech.) in Computer Science & Engineering **Graduated 2017**

Mar Athanasius College of Engineering (MACE) *Kerala, India*

- Completed B.Tech in CSE with high academic standing.

Technical, AI & Academic Projects

DTwin — Building Digital Twin with an AI Copilot **Personal Project, built with Claude Code**

- **Digital Twin Platform:** Demo office building on simulated telemetry — PostGIS + TimescaleDB, Node.js ingest with WebSocket fan-out, FastAPI energy simulator, react-three-fiber 3D dashboard; multi-tenant via row-level security; expiring setpoint overrides checked by a safety envelope.
- **Human-Approved AI Copilot:** LangGraph.js + Claude agent that cannot change a setpoint without human approval — its command node is reachable only via an `interrupt()`, asserted by a graph test; 293 end-to-end checks in CI.

GIS-Integrated Water Network Database System **M.Tech Project – IIT Bombay**

- Designed and developed a GIS-integrated water network database application in QGIS using Python and SQL. Programmed custom Python plugins to automate node elevation extraction from contour files and web APIs, and built efficient data extraction workflows for EPANET and JalTantra.

Teaching Assistantships

Graduate Teaching Assistant **Jul 2018 – Nov 2019**

Indian Institute of Technology (IIT), Bombay *Mumbai, India*

- **Software Lab (CS699):** Guided M.Tech and Ph.D. students in programming assignments, managed lab sessions, and performed continuous evaluation and grading of lab work under Prof. Kavi Arya.
- **Computer Programming (CS101):** Led lab sections for 400+ undergraduate students in a team of 45+ TAs under Prof. Om P. Damani and Prof. Ganesh Ramakrishnan; mentored 14 students and evaluated exams.